Task 9 – Implement and Test the Calculator

Python Programming • Assignment 1 • Village Milk Co-operative Payment Calculator | Marks: 4 | Bloom's level: K3 – Apply

Name: **Dhaarani D** Register No.: _____ GitHub: _____ (folder Task09)

1. Working source code (milkcalc.py)

Design choices from Tasks 1–8: money and readings are **Decimal** built from strings; all rates are named constants at the top; steps use round-half-up; every entry is validated; all calculation functions return values and only the demonstration at the bottom prints.

milkcalc.py

```
"""Milk co-operative payment calculator (final design: Decimal money, validated input)."""
from decimal import Decimal, InvalidOperation, ROUND_HALF_UP

# ---- rate card: the ONLY place where rates are typed ----
BASE_RATE = Decimal("30.00")      # Rs per litre at standard quality
FAT_STEP_VALUE = Decimal("0.40")  # Rs per 0.1 % fat
SNF_STEP_VALUE = Decimal("0.30")  # Rs per 0.1 % SNF
STD_FAT, STD_SNF = Decimal("3.5"), Decimal("8.5")
STEP = Decimal("0.1")
MIN_FAT, MAX_FAT = Decimal("2.0"), Decimal("7.0")
MIN_SNF, MAX_SNF = Decimal("6.0"), Decimal("10.0")   # assumption
MAX_LITRES = Decimal("200")                          # assumption: per entry
PAISA = Decimal("0.01")


def to_money(value):
    """Rounds a Decimal to whole paise (half up, as banks do)."""
    return value.quantize(PAISA, rounding=ROUND_HALF_UP)


def parse_number(text, name):
    """Converts text to Decimal or raises ValueError with a clear message."""
    try:
        value = Decimal(text.strip())
    except InvalidOperation:
        raise ValueError(f"{name}: '{text}' is not a number") from None
    if not value.is_finite():
        raise ValueError(f"{name}: '{text}' is not a finite number")
    return value


def validate_entry(litres, fat, snf):
    """Checks the business rules; raises ValueError if any rule is broken."""
    if not (0 < litres <= MAX_LITRES):
        raise ValueError(f"litres must be above 0 and at most {MAX_LITRES}")
    if not (MIN_FAT <= fat <= MAX_FAT):
        raise ValueError(f"fat must be between {MIN_FAT} and {MAX_FAT}")
    if not (MIN_SNF <= snf <= MAX_SNF):
        raise ValueError(f"snf must be between {MIN_SNF} and {MAX_SNF}")


def rate_per_litre(fat, snf):
    """Price per litre in Decimal rupees (2 decimals)."""
    fat_steps = ((fat - STD_FAT) / STEP).to_integral_value(rounding=ROUND_HALF_UP)
    snf_steps = ((snf - STD_SNF) / STEP).to_integral_value(rounding=ROUND_HALF_UP)
    return to_money(BASE_RATE + fat_steps * FAT_STEP_VALUE + snf_steps * SNF_STEP_VALUE)


def payout(litres, fat, snf):
    """Amount payable for one entry, rounded to the paisa."""
    validate_entry(litres, fat, snf)
    return to_money(litres * rate_per_litre(fat, snf))


def ten_day_total(entries):
    """Total of payouts for a list of (litres, fat, snf) Decimal tuples."""
    total = Decimal("0.00")
    for litres, fat, snf in entries:
        total += payout(litres, fat, snf)
```

```
    return total


def read_entry(l_text, f_text, s_text):
    """Parses three strings and returns the payout (ValueError on bad input)."""
    litres = parse_number(l_text, "litres")
    fat = parse_number(f_text, "fat")
    snf = parse_number(s_text, "snf")
    return payout(litres, fat, snf)


if __name__ == "__main__":
    print("Farmer  Litres   Fat   SNF     Rate      Amount")
    for fid, (l, f, s) in {"F001": ("10", "3.5", "8.5"), "F002": ("8.5", "4.2", "8.7"),
                           "F003": ("12", "3.1", "8.3")}.items():
        L, F, S = Decimal(l), Decimal(f), Decimal(s)
        print(f"{fid}  {L:7} {F:5} {S:5} {rate_per_litre(F, S):8.2f} {payout(L, F, S):10.2f}")
```

python milkcalc.py

```
Farmer  Litres   Fat   SNF     Rate      Amount
F001       10   3.5   8.5    30.00     300.00
F002      8.5   4.2   8.7    33.40     283.90
F003       12   3.1   8.3    27.80     333.60
```

2. Test program (assert-based)

test_milkcalc.py

```
from decimal import Decimal as D
import milkcalc as mc

results = []

def check(label, expected, actual):
    ok = expected == actual
    results.append(ok)
    print(f"{label:<38}{str(expected):>10}{str(actual):>10}  {'PASS' if ok else 'FAIL'}")
    assert ok, label            # stops at the first failure

def verdict(l, f, s):
    try:
        mc.read_entry(l, f, s)
    except ValueError:
        return "rejected"
    return "accepted"

print(f"{'Test case':<38}{'Expected':>10}{'Actual':>10}  Result")
# --- normal entries (above and below the standard) ---
check("N1 fat+ snf+  8.5 L 4.2/8.7", D("283.90"), mc.payout(D("8.5"), D("4.2"), D("8.7")))
check("N2 fat- snf-  12 L 3.1/8.3",  D("333.60"), mc.payout(D("12"), D("3.1"), D("8.3")))
check("N3 fat+ snf-  10 L 4.0/8.2",  D("311.00"), mc.payout(D("10"), D("4.0"), D("8.2")))
check("N4 fat- snf+  5 L 3.0/8.8",   D("144.50"), mc.payout(D("5"), D("3.0"), D("8.8")))
# --- boundary entries ---
check("B1 standard   10 L 3.5/8.5",  D("300.00"), mc.payout(D("10"), D("3.5"), D("8.5")))
check("B2 lowest fat 10 L 2.0/8.5",  D("240.00"), mc.payout(D("10"), D("2.0"), D("8.5")))
check("B3 highest fat 10 L 7.0/8.5", D("440.00"), mc.payout(D("10"), D("7.0"), D("8.5")))
check("B4 smallest qty 0.1 L 3.5/8.5", D("3.00"), mc.payout(D("0.1"), D("3.5"), D("8.5")))
check("B5 half-paise 0.05 L 3.6/8.4", D("1.51"), mc.payout(D("0.05"), D("3.6"), D("8.4")))
# --- invalid entries (all must be rejected) ---
for label, args in [("I1 text litres 'eight'", ("eight", "3.5", "8.5")),
                    ("I2 negative litres '-5'", ("-5", "3.5", "8.5")),
                    ("I3 zero litres '0'", ("0", "3.5", "8.5")),
                    ("I4 fat above range 7.1", ("10", "7.1", "8.5")),
                    ("I5 fat below range 1.9", ("10", "1.9", "8.5")),
                    ("I6 comma decimal '8,5'", ("10", "3.5", "8,5")),
                    ("I7 'nan' as fat", ("10", "nan", "8.5")),
                    ("I8 snf above range 11", ("10", "3.5", "11"))]:
    check(label, "rejected", verdict(*args))
# --- hand-calculated set of 5 entries ---
five = [(D("10"), D("3.5"), D("8.5")), (D("8.5"), D("4.2"), D("8.7")), (D("12"), D("3.1"), D("8.3")),
        (D("10"), D("4.0"), D("8.2")), (D("5"), D("3.0"), D("8.8"))]
check("H1 five-entry total (by hand)", D("1373.00"), mc.ten_day_total(five))
print(f"\n{sum(results)} of {len(results)} tests passed")
```

3. Test table – expected and actual results

python test_milkcalc.py

```
Test case                              Expected    Actual  Result
N1 fat+ snf+  8.5 L 4.2/8.7             283.90    283.90  PASS
N2 fat- snf-  12 L 3.1/8.3              333.60    333.60  PASS
N3 fat+ snf-  10 L 4.0/8.2              311.00    311.00  PASS
N4 fat- snf+  5 L 3.0/8.8               144.50    144.50  PASS
B1 standard   10 L 3.5/8.5              300.00    300.00  PASS
B2 lowest fat 10 L 2.0/8.5              240.00    240.00  PASS
B3 highest fat 10 L 7.0/8.5             440.00    440.00  PASS
B4 smallest qty 0.1 L 3.5/8.5             3.00      3.00  PASS
B5 half-paise 0.05 L 3.6/8.4              1.51      1.51  PASS
I1 text litres 'eight'                rejected  rejected  PASS
I2 negative litres '-5'               rejected  rejected  PASS
I3 zero litres '0'                    rejected  rejected  PASS
I4 fat above range 7.1                rejected  rejected  PASS
I5 fat below range 1.9                rejected  rejected  PASS
I6 comma decimal '8,5'                rejected  rejected  PASS
I7 'nan' as fat                       rejected  rejected  PASS
I8 snf above range 11                 rejected  rejected  PASS
H1 five-entry total (by hand)          1373.00   1373.00  PASS

18 of 18 tests passed
```

Group	Cases	What it shows
Normal (N1–N4)	fat/SNF above and below the standard in all four combinations	The step rule works in both directions (283.90, 333.60, 311.00, 144.50).
Boundary (B1–B5)	exactly 3.5/8.5; lowest fat 2.0; highest fat 7.0; smallest quantity; a half-paisa tie	Limits accepted; 0.05 L × Rs 30.10 = 1.505 is paid as 1.51 (round half up).
Invalid (I1–I8)	text, negative, zero, fat above/below the range, comma decimal, nan, SNF out of range	Every invalid entry is rejected with ValueError, none is paid.
Hand total (H1)	5 entries: 300.00 + 283.90 + 333.60 + 311.00 + 144.50	Calculated by hand as 1,373.00; program gives 1373.00.

Alternative Concept and Complexity Analysis

Alternative: test by typing inputs by hand each time and reading the answer on the screen.

	Manual typing	assert-based test function
Process	type litres, fat and SNF for each case, compare with the hand answer by eye, repeat after every change	write each case once with its expected value; run one command; a failing assert stops with the case name
Time to run all 18 cases	≈ 18 × 30 s ≈ 9 minutes (at about 30 s per case), i.e. O(m) human time for m cases, repeated after every change	≈ 22 ms including starting Python; O(m) machine time with a tiny constant
Reliability	a tired tester skips or misreads cases	identical every time; also records the expected value permanently
Rerun after each of c changes	≈ 9 min × c	≈ 0.02 s × c

Both grow linearly with the number of cases m, but the constant differs by a factor of several thousand, so only automated tests are practical when the code is changed repeatedly (rate revisions, new rules).

Evidence

Working source code and a test table with expected and actual results for every case (18 of 18 passed). Source: Task09/milkcalc.py, test_milkcalc.py.